# Smart Agriculture IoT Alert System

Architectural Specification & Mathematical Foundations

**Abstract:** This document outlines the end-to-end architecture of a modern predictive maintenance pipeline for agriculture. The system ingests raw IoT sensor data, models temporal dependencies using a Deep Learning LSTM, and leverages a local Generative AI agent (Llama 3.2) to translate deterministic forecasts into human-readable alerts via a Streamlit dashboard.

## Phase 1: Data Ingestion & Preparation (data_prep.py)

Raw IoT data is inherently volatile. Sensors experience downtime, and the scale of different measurements (e.g., temperature vs. humidity) varies wildly. The data must be structured into overlapping tensors before it can be fed into a neural network.

### Step 1.1: Handling Missing Data

Real-world datasets, such as `SmartFarm.csv`, often contain NaN values where sensors dropped packets. We utilized the Pandas `.ffill()` (Forward Fill) method. This technique propagates the last known valid sensor reading forward, maintaining the integrity of the chronological timeline without introducing synthetic statistical artifacts (such as a mean or median calculation might).

### Step 1.2: Feature Scaling

Deep learning models optimize their weights via gradient descent. If humidity is in the 80s and temperature is in the 20s, the larger magnitude of humidity will disproportionately dominate the network's weight updates, causing the model to learn unevenly. We applied Scikit-Learn's `MinMaxScaler` to crush all features into a uniform range of $[0, 1]$.

**The Mathematics of Scaling:** For every value $X$ in a given column, the scaled value is computed as:

$$X_{scaled} = (X - X_{min}) / (X_{max} - X_{min})$$

### Step 1.3: Sequential Windowing (Sliding Windows)

Neural networks cannot forecast time-series data from a single, static row. The network requires a "historical context" window to identify trends. We established a `seq_length` of 24 (representing 24 hours). We iteratively sliced the scaled dataset into overlapping 3D arrays to define the input features ($X$) and the corresponding target vector ($y$).

## Phase 2: Deep Learning Architecture (train_lstm.py)

This module houses the core predictive engine. We utilized TensorFlow and Keras to construct a Long Short-Term Memory (LSTM) network, an architecture specifically designed to retain long-term dependencies across time steps while avoiding the vanishing gradient problem common in standard recurrent networks.

### Step 2.1: Chronological Train/Test Split

We split the processed sequences into an 80% training set and a 20% testing set. Unlike standard machine learning tasks, we explicitly disabled shuffling (`shuffle=False`). Randomizing time-series data induces "data leakage," wherein the model inadvertently uses future data points (e.g., Friday's temperature) to train its predictions for the past (e.g., Thursday). This strictly invalidates the model's predictive integrity.

### Step 2.2: The LSTM Layer (50 Units)

The first layer of our `Sequential` model is an LSTM tasked with learning the temporal patterns of the 3 features across the 24-hour window.

**Mathematical Reasoning (Why 50 Units?):** The `units` parameter dictates the dimensionality of both the hidden state vector ($h_t$) and the internal cell state memory ($C_t$). We set this to 50.

$$h_t \in \mathbb{R}^{50}$$

Setting the units to 50 provides sufficient neural capacity (parameters) to map complex, non-linear environmental oscillations (e.g., morning dew cycles, rapid afternoon temperature spikes) without risking heavy overfitting, which often occurs if units exceed 128 for simple 3-feature datasets.

### Step 2.3: The Dense Output Layer (3 Units)

The LSTM outputs a 50-dimensional hidden state vector at the final time step. Because our objective is to predict exactly three target variables (Moisture, Temperature, Humidity), we appended a standard fully-connected `Dense` layer.

**Mathematical Reasoning:** The Dense layer performs a linear transformation, applying a learned weight matrix ($W$) and bias vector ($b$) to reduce the 50 dimensions down to 3.

$$Y = W \cdot h_t + b \quad \textit{where} \quad Y \in \mathbb{R}^3$$

## Phase 3: Generative AI Bridge (llm_agent.py)

To transition the system from a pure mathematical model into an actionable tool, we integrated a local Large Language Model (LLM) to act as an "Incident Commander." Agricultural operators require immediate, contextual insights, not an array of scaled floats.

### Step 3.1: Prompt Engineering & Data Injection

We authored a system prompt explicitly defining the AI's persona. Using Python f-strings, the raw float predictions outputted by the LSTM are dynamically injected into the prompt. The model is instructed to analyze these three values and return a 2-3 sentence diagnostic brief.

### Step 3.2: Local LLM Routing (Ollama)

We leveraged the `requests` library to send a POST payload directly to the local Ollama API endpoint (`http://localhost:11434/api/generate`) targeting the **Llama 3.2** model. Performing this inference locally ensures zero latency costs, zero API billing, and complete data privacy for proprietary agricultural metrics.

## Phase 4: Production Deployment (dashboard.py)

The entire pipeline is wrapped in an interactive, event-driven Streamlit dashboard, providing the end-user with live historical data and automated future insights.

| Deployment Step | Technical Implementation & Reasoning |
| --- | --- |
| **Resource Caching** | Utilized Streamlit's `@st.cache_resource` decorator when loading the TensorFlow `.keras` model. Loading deep learning models into VRAM is highly computationally expensive; caching ensures the model is loaded only once upon initialization, preventing UI freezes during user interaction. |
| **Inference Slicing** | When triggered, the system extracts the literal last 24 rows of the historical CSV, scales them, and reshapes them into a single tensor block of shape `(1, 24, 3)` to feed the LSTM. |

### Step 4.3: Inverse Transformation Logic

Because the LSTM was trained on scaled data $[0, 1]$, its output prediction is also scaled. Before we can display these numbers to the user or pass them to the LLM, we must reverse the normalization utilizing the saved `MinMaxScaler` object.

**The Mathematics of Inverse Scaling:**